# Introductory Econometrics with Recitation: TA Session Week 12

Danny Po-Hsien Kang (康柏賢)

May 11, 2026

# Today's agenda

1. Data Cleaning
   - `pivot_longer()`
   - `pivot_wider()`
2. Logarithmic Regression Models
   - Model Setup
   - Coefficient Interpretation
   - Prediction
3. Binary Regression Models
   - Conditional Expectation Function
   - LPM, Probit, and Logit
   - Prediction
   - Pseudo R-squared

# Today's Dataset

- Today we will use the following CSV file from the OpenIntro Website:
  - `acs12.csv`: Results from the US Census American Community Survey, 2012.
- Please import the dataset into RStudio.

```
acs <- read.csv("data/acs12.csv")

acs <- acs %>%
  na.omit() %>%
  mutate(married = ifelse(married == "yes", 1, 0)) %>%
  filter(income != 0)
```

# Data Cleaning: pivot_longer()

- pivot_longer() turns several columns into two columns:
  - one column stores the old column names
  - one column stores the old cell values
- It is useful when the same variable is spread across many columns.
- **Syntax:**

```
data_long <- data_wide %>%
  pivot_longer(
    cols = c(col_1, col_2, col_3),
    names_to = "variable_name",
    values_to = "value"
  )
```

# Data Cleaning: pivot_longer()

- **Example:**

```
score_wide <- data.frame(
  id = c(1, 2),
  quiz1 = c(80, 90),
  quiz2 = c(85, 95)
)

score_long <- score_wide %>%
  pivot_longer(
    cols = c(quiz1, quiz2),
    names_to = "quiz",
    values_to = "score"
  )
```

# Data Cleaning: pivot_longer()

- Each student now has one row for each quiz.
- **Output:**

```
  student quiz   score
1 Alice   quiz_1    80
2 Alice   quiz_2    85
3 Bob     quiz_1    90
4 Bob     quiz_2    95
```

# Data Cleaning: pivot_wider()

- pivot_wider() does the opposite of pivot_longer().
- It turns values in two columns into several new columns.
- **Syntax:**

```
data_wide <- data_long %>%
  pivot_wider(
    names_from = "variable_name",
    values_from = "value"
  )
```

- It is useful when each unit should appear in one row.

# Data Cleaning: pivot_wider()

- **Example:**

```
score_wide_again <- score_long %>%
  pivot_wider(
    names_from = quiz,
    values_from = score
  )
```

- **Output:**

```
  student quiz_1 quiz_2
1 Alice        80     85
2 Bob          90     95
```

- In practice, choose the format that makes later analysis easier.

# Logarithmic Regression Models

- Consider a basic linear model:

$$\text{income}_i = \beta_0 + \beta_1 \text{hrs\_work}_i + \varepsilon_i.$$

- In this model, a one-hour increase in `hrs_work` is associated with a $\beta_1$ dollar increase in predicted income.
- To allow nonlinear relationships, we can use logarithms:

$$\begin{aligned}
\text{income}_i &= \beta_0 + \beta_1 \log(\text{hrs_work}_i) + \varepsilon_i, \\
\log(\text{income}_i) &= \beta_0 + \beta_1 \text{hrs_work}_i + \varepsilon_i, \\
\log(\text{income}_i) &= \beta_0 + \beta_1 \log(\text{hrs_work}_i) + \varepsilon_i.
\end{aligned}$$

# Logarithmic Regression Models

- First, create the log variables.

```
acs <- acs %>%
  mutate(
    lnincome = log(income),
    lnhrs_work = log(hrs_work)
  )
```

- Then estimate the four models.

```
model1 <- lm_robust(income ~ hrs_work, data = acs)
model2 <- lm_robust(income ~ lnhrs_work, data = acs)
model3 <- lm_robust(lnincome ~ hrs_work, data = acs)
model4 <- lm_robust(lnincome ~ lnhrs_work, data = acs)
```

# Logarithmic Regression Models

- **Output:**

| | Linear | Linear-Log | Log-Linear | Log-Log |
|---|---|---|---|---|
| (Intercept) | -16905.31 * | -101685.91 *** | 7.87 *** | 3.74 *** |
| | (7511.36) | (17372.28) | (0.18) | (0.39) |
| hrs_work | 1641.66 *** | | 0.06 *** | |
| | (209.69) | | (0.00) | |
| lnhrs_work | | 41452.42 *** | | 1.79 *** |
| | | (4993.46) | | (0.11) |
| R^2 | 0.12 | 0.09 | 0.37 | 0.42 |
| Adj. R^2 | 0.12 | 0.09 | 0.37 | 0.42 |
| Num. obs. | 745 | 745 | 745 | 745 |
| RMSE | 55083.74 | 55838.30 | 0.96 | 0.92 |

*** p < 0.001; ** p < 0.01; * p < 0.05

# Coefficient Interpretation for Different Models

| Model | Change in $X$ | Resulting change in $\widehat{Y}$ |
|---|---|---|
| Linear | 1 unit | $\beta_1$ |
| Linear-Log | 1% | $0.01 \times \beta_1$ |
| Log-Linear | 1 unit | $(100 \times \beta_1)\%$ |
| Log-Log | 1% | $\beta_1\%$ |

- The interpretation depends on whether $Y$, $X$, or both are in logs.
- For log models, the interpretation is often about percentage changes.

# Coefficient Interpretation: Examples

- Suppose the estimated coefficient in the Linear-Log model is $41452.42$.

$$1\% \text{ increase in hrs\_work} \Rightarrow 0.01 \times 41452.42 = 414.52$$

- Income is predicted to increase by about $414.52$ dollars.
- Suppose the estimated coefficient in the Log-Log model is $1.79$.

$$1\% \text{ increase in hrs\_work} \Rightarrow 1.79\% \text{ increase in predicted income.}$$

# Prediction with Logarithmic Models

- Suppose a worker increases working hours from 40 to 42.

```
low_hr <- data.frame(
  hrs_work = 40,
  lnhrs_work = log(40)
)

high_hr <- data.frame(
  hrs_work = 42,
  lnhrs_work = log(42)
)
```

- This is also approximately a 5% increase in working hours.

# Prediction with Logarithmic Models

- For the Linear model:

```
predict(model1, high_hr) - predict(model1, low_hr) # 3283.326
```

- For the Linear-Log model:

```
predict(model2, high_hr) - predict(model2, low_hr) # 2022.47
```

- For Log-Linear and Log-Log models, predictions are on the log scale.

```
exp(predict(model3, high_hr)) - exp(predict(model3, low_hr))
# 3569.997
exp(predict(model4, high_hr)) - exp(predict(model4, low_hr))
# 2905.476
```

# Conditional Expectation Function

- The conditional expectation function is:

$$E[Y \mid X] = \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k.$$

- If $Y$ is binary, then:

$$E[Y \mid X] = P(Y = 1 \mid X).$$

- In this case, OLS gives the linear probability model (LPM):

$$P(Y = 1 \mid X) = \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k.$$

# Why Not Only Use LPM?

- The LPM is easy to estimate and interpret.
- However, it may predict probabilities smaller than 0 or larger than 1.
- To keep predicted probabilities between 0 and 1, we can use nonlinear models.
- Two common models are:
  - Logit model
  - Probit model

# Logit and Probit Models

- The Logit model is:

$$P(Y = 1 \mid X) = \frac{\exp(\beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k)}{1 + \exp(\beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k)}.$$

- The Probit model is:

$$P(Y = 1 \mid X) = \Phi(\beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k),$$

where $\Phi$ is the standard normal CDF.

# Binary Regression: LPM

- We use `lm_robust()` to estimate the linear probability model.
- **Syntax:**

```
lpm <- lm_robust(
  married ~ age,
  data = acs
)
```

# Binary Regression: Probit and Logit

- We use `glm()` with `family = binomial()` to estimate logit and probit models.
- **Syntax:**

```
logit <- glm(
  married ~ age,
  data = acs,
  family = binomial(link = "logit")
)

probit <- glm(
  married ~ age,
  data = acs,
  family = binomial(link = "probit")
)
```

# Binary Regression: Comparison

- **Model comparison:**

```
screenreg(
  list(lpm, logit, probit),
  custom.model.names = c("LPM", "Logit", "Probit")
)
```

- Coefficient interpretation is different across the three models.

# Binary Regression: Comparison

- **Output:**

| | LPM | Logit | Probit |
|---|---|---|---|
| (Intercept) | 0.08 | -1.93 *** | -1.16 *** |
| | (0.05) | (0.26) | (0.15) |
| age | 0.01 *** | 0.05 *** | 0.03 *** |
| | (0.00) | (0.01) | (0.00) |
| R^2 | 0.12 | | |
| Adj. R^2 | 0.12 | | |
| Num. obs. | 745 | 745 | 745 |
| RMSE | 0.46 | | |
| AIC | | 923.09 | 925.14 |
| BIC | | 932.32 | 934.37 |
| Log Likelihood | | -459.55 | -460.57 |
| Deviance | | 919.09 | 921.14 |

*** p < 0.001; ** p < 0.01; * p < 0.05

# Binary Regression: Prediction

- Create two new observations.

```
young <- data.frame(age = 25)
old <- data.frame(age = 65)
```

- For LPM, use `predict()` directly.

```
predict(lpm, young)
```

- For logit and probit models, add `type = "response"` to get predicted probabilities.

```
predict(logit, young, type = "response")
predict(probit, young, type = "response")
```

# Binary Regression: Pseudo R-squared

- For logit and probit models, we usually use pseudo $R^2$ instead of the usual $R^2$.
  - Use `pR2()` from the `pscl` package.

```
library(pscl)
pR2(logit)
```

```
fitting null model for pseudo-r2
    llh   llhNull       G2    McFadden     r2ML     r2CU
-459.55   -506.85    94.61        0.09     0.12     0.16
```

- `llh`: log-likelihood of the fitted model.
- `llhNull`: log-likelihood of the null model.
- `McFadden`: McFadden's pseudo $R^2$.